A Deep Learning–Based Responsive Web Application for Tomato Leaf Disease Diagnosis Using MobileNetV3Small

Aldrea Timotei Sarmiento[#1], Marc Miranda[#2], Charlene De Chavez[#3]

[#]*Bachelor of Science in Computer Science, National University*
Lipa City Batangas, Philippines

[1]sarmientoax3@students.nu-lipa.edu.ph

[2]mirandamf@students.nu-lipa.edu.ph

[3]dechavezct@students.nu-lipa.edu.ph

Abstract– This study presents the development of a real-time tomato leaf disease classification system using MobileNetV3Small, a lightweight convolutional neural network optimized for mobile and web deployment. The model was trained on a balanced dataset of 7,500 images covering five tomato leaf conditions: Bacterial Spot, Healthy, Leaf Mold, Septoria Leaf Spot, and Yellow Leaf Curl Virus. A developmental-experimental quantitative research design guided the process, combining transfer learning and fine-tuning strategies to enhance model accuracy and efficiency.

Two training configurations were tested: a baseline (non-fine-tuned) and a fine-tuned model. The fine-tuned version achieved a validation accuracy of 90%, surpassing the baseline's 80%, with improved precision, recall, and F1-score values of 90%. Training performance, confusion matrices, and Grad-CAM visualizations were used to assess learning behavior and model interpretability. The final model was integrated into a cross-platform system accessible via both web and mobile applications, enabling real-time disease diagnosis.

Results demonstrate the effectiveness of fine-tuning and early stopping in improving classification accuracy while maintaining deployment efficiency. The system offers a promising tool for smart agriculture, providing reliable and accessible tomato disease diagnosis for farmers and agricultural practitioners.

Keywords– Tomato Leaf Disease, MobileNetV3Small, Image Classification, Deep Learning, Web Application, Precision Agriculture

I. INTRODUCTION

Tomato (Solanum lycopersicum) is one of the world's most economically significant vegetable crops, contributing to global food security and the livelihood of many smallholder farmers. Despite its importance, tomato cultivation is highly susceptible to various foliar diseases that impact both yield and quality. Manual identification of these diseases is often slow, labor-intensive, and error-prone without expert input, making timely and accurate diagnosis essential for effective management and minimizing economic loss.

Among the most prevalent diseases are Early Blight (Alternaria solani), characterized by dark, concentric lesions on older leaves; Late Blight (Phytophthora infestans), which spreads rapidly in humid conditions and presents as white, fuzzy patches on the underside of leaves; and Yellow Leaf Curl Virus (TYLCV), which causes yellowing, curling, stunting, and reduced leaf size. TYLCV alone can cause yield losses of up to 90–100%, affecting nearly 7 million hectares globally (Zhang et al., 2022; Sánchez-Campos et al., 2021). Septoria Leaf Spot (Septoria lycopersici) is marked by small dark spots with tan centers and black specks and is particularly damaging during periods of high moisture (Gugino et al., 2018; Ivey, 2023). Other common diseases include Leaf Mold (Passalora fulva) and Bacterial Spot (Xanthomonas), which also significantly affect plant health. For comparison and classification, healthy leaves are also considered as reference samples.

The rise of deep learning has significantly improved the accuracy and speed of plant disease detection. CNN-based models such as AlexNet, VGG, and DenseNet have demonstrated high classification performance, with AlexNet achieving 97.49% accuracy (Rangarajan et al.), and DenseNet reaching 99.75% (Mahmud et al., 2024). Durmus et al. reported 95.65% accuracy using AlexNet on the PlantVillage dataset. Further innovations include the integration of MobileNetV2 with ResNet50, enhanced with Batch Normalization and Dropout layers, as well as

advanced segmentation models like U-Net and MU-Net, the latter achieving 98.66% accuracy, 98.5% IoU, and 98.73% dice coefficient. Hybrid models combining traditional machine learning with deep learning have also emerged, including seven Bayesian-optimized systems for classifying ten tomato leaf types (Hukkeri et al., 2024).

Despite these advances, challenges remain. High computational requirements, environmental noise, and poor lesion visibility limit the real-world applicability of many DL models. Still, the integration of these technologies with agricultural practices offers great potential for early disease detection, reducing pesticide misuse, and improving yield sustainability.

This study proposes a web-based tomato disease classification system using a two-stage MobileNetV3-S CNN architecture. The first model verifies whether the uploaded image is a tomato leaf, while the second classifies it as healthy or identifies one of several common diseases, including Yellow Leaf Curl Virus, Leaf Mold, Bacterial Spot, or Septoria Leaf Spot. This dual approach enhances accuracy by filtering irrelevant inputs before classification.

Aligned with the United Nations Sustainable Development Goals, this research supports SDG 2: Zero Hunger by improving agricultural productivity through early disease detection, SDG 9: Industry, Innovation, and Infrastructure by applying AI in rural agriculture, and SDG 12: Responsible Consumption and Production by promoting data-informed pesticide management. By providing an accessible diagnostic tool, the system empowers smallholder farmers—especially in the Philippines—to make timely, informed decisions, supporting a more resilient and sustainable agricultural sector.

II. METHODOLOGY

A. Research Design

This study employs a developmental-experimental quantitative research design to build and evaluate a deep learning-based tomato leaf disease classification system. The developmental phase focused on designing a lightweight convolutional neural network using MobileNetV3Small, trained on a balanced dataset with 1,500 images per class across five categories: Bacterial Spot, Healthy, Leaf Mold, Septoria Leaf Spot, and Yellow Leaf Curl Virus. Transfer learning was applied to fine-tune the model, optimizing it for accuracy and efficiency.

The experimental phase involved integrating the trained model into a cross-platform application accessible via both web and mobile. Users can upload leaf images to receive real-time diagnostic feedback, making the system practical for field use, especially in low-resource environments.

To assess effectiveness, the model was evaluated using accuracy, precision, recall, and F1-score, while the system's usability was informally tested across platforms. This combined approach ensures both technical performance and real-world applicability, contributing to smart and sustainable agricultural practices.

B. System Architecture

This activity diagram illustrates the full workflow of a web-based plant disease prediction platform that processes user-submitted leaf images using a combination of image processing techniques, a trained deep learning model, and a local large language model (LLM). The system is designed to deliver accurate diagnoses along with easy-to-understand explanations.

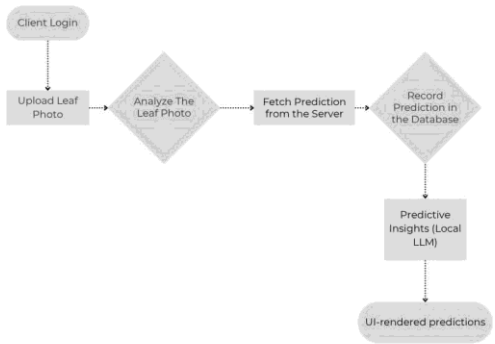

Fig. 1 Activity Diagram

The process begins when a user securely logs into the web platform using their credentials. Once logged in, the user is taken to an interface where they can upload an image of a plant leaf. The system verifies that the uploaded image meets quality requirements—such as appropriate resolution, file type, and clarity—before moving on.

After passing the initial validation check, the image undergoes preprocessing steps. These include resizing the image to a standard input size, normalizing pixel values, and performing any necessary color adjustments. This ensures that the image is properly formatted for analysis by the deep learning model.

Next, the preprocessed image is analyzed using a trained convolutional neural network (CNN) model. This model has been trained to recognize various plant diseases based on visual features in the leaf. It extracts relevant

characteristics such as texture, shape, and color patterns to make a prediction about the presence and type of disease.

The extracted features and initial prediction are then sent to a remote server that hosts a more powerful version of the model. This backend system performs a secondary analysis to validate or refine the prediction, ensuring high accuracy even in complex or ambiguous cases.

Once a final prediction is generated, it is stored in a centralized database. This allows the system to keep a record of all user submissions and their corresponding results, which can be accessed later for review or further analysis.

To enhance the user experience, the raw prediction is passed through a locally hosted large language model (LLM). The LLM interprets the technical output and generates a user-friendly explanation. This may include a summary of the identified disease, its typical symptoms, potential causes, and basic care or treatment recommendations.

Finally, the interpreted results are displayed on the user interface in a clear and accessible format. The interface is designed to be easy to navigate, allowing users to understand their leaf diagnosis at a glance and take appropriate action based on the system's recommendations.

This end-to-end system ensures that users—from farmers to hobbyist gardeners—receive fast, accurate, and actionable insights from a simple photo upload.

C. Data Collection

The dataset used in this study comprises a total of 7,000 tomato leaf images distributed evenly across five categories: Bacterial Spot, Healthy, Leaf Mold, Septoria Leaf Spot, and Yellow Leaf Curl Virus, with 1,500 images per class. The images were collected from a combination of sources, including publicly available datasets from Kaggle and locally curated images gathered in field conditions. In addition to the main dataset, a separate, locally sourced test set was prepared, containing 100 images per class, totaling 500 test images, which were used exclusively to assess the model's final performance in real-world conditions.

TABLE 1

MODEL DATASET

Class	Training/Validation Images	Test Images
Bacterial Spot	1,500	100
Healthy	1,500	100
Leaf Mold	1,500	100
Septoria Leaf Spot	1,500	100
Yellow Leaf Curl Virus	1,500	100
Total	**7,500**	**500**

All images were preprocessed by resizing them to 256×256 pixels, the chosen input dimension compatible with the MobileNetV3Small architecture. This resizing ensures that input images are consistent and optimized for GPU training. After resizing, the images were normalized by scaling pixel values between 0 and 1, which helps improve the learning rate and training stability of the neural network.

To increase model robustness and generalization, data augmentation was applied to the training set using a series of transformations. These augmentations were implemented dynamically during data loading using TensorFlow's Keras preprocessing layers. The applied transformations include random cropping, rotation, flipping, brightness and contrast adjustment, zooming, and translation. These simulate various real-world image capture scenarios such as off-centered framing, poor lighting, or partial occlusion, thus making the model more adaptable to unpredictable user inputs.

TABLE 2

IMAGE AUGMENTATION TECHNIQUES

Augmentation Type	Description
Random Crop	Simulates partial leaf visibility
Random Rotation	Rotates the image up to 50%
Random Flip	Applies horizontal and vertical flipping
Random Brightness	Adjusts brightness in the range [0.1, 0.6]
Random Contrast	Modifies image contrast by up to 0.4
Random Zoom	Zooms in/out by up to 30%
Random Translation	Shifts image position by 20% horizontally and vertically

The main dataset was divided into an 80-20 split, assigning 80% for training and 20% for validation. The full augmentation pipeline was applied exclusively to the training data. The independent test set, collected locally and completely unseen during training, was also kept augmentation-free to ensure realistic performance assessment.

This preprocessing strategy ensured that the model was trained not only on a diverse and balanced dataset but also

exposed to a wide range of image variations that mimic real-world use, enhancing both its accuracy and adaptability when deployed.

D. Development Environment

The experimentation and model development in this study were conducted using Python as the primary programming language, chosen for its robust ecosystem in machine learning and deep learning. The models were built, trained, and evaluated using Keras with a TensorFlow backend, which provided a high-level interface for efficient and modular deep learning development. All processes were executed in a local environment configured through Anaconda, which facilitated effective package and environment management. Visual Studio Code (VS Code) served as the primary code editor due to its flexibility and integration capabilities. This setup ensured a stable, organized, and reproducible development workflow.

D. Model Development

To accurately classify tomato leaf diseases into five specific categories, the researchers developed a deep learning model based on the MobileNetV3Small architecture. MobileNetV3Small is a compact and efficient convolutional neural network optimized for mobile and web deployment, making it well-suited for real-time agricultural diagnostics. The model was initialized with pretrained weights from the ImageNet dataset, allowing it to benefit from rich visual feature representations without requiring extensive training from scratch.

The pretrained base model was imported with its top classification layer removed (include_top=False) and configured to accept input images of shape 256 × 256 × 3. To this convolutional base, a custom classifier head was appended to adapt the model to the specific classification task. The classifier consisted of multiple fully connected (dense) layers with ReLU activations, as well as Batch Normalization and Dropout layers to improve learning stability and reduce overfitting. The final output layer used a softmax activation function to generate class probabilities for the five disease categories.

Initially, all layers in the MobileNetV3Small base were frozen, meaning their weights were not updated during early training. This approach preserved the learned features from the ImageNet dataset. After initial training, fine-tuning was applied by unfreezing the top 80 layers of the base model to allow further learning specific to tomato leaf characteristics. This two-phase training strategy allowed the model to efficiently combine general visual understanding with domain-specific refinement.

The complete model architecture is summarized in Table 3.

TABLE 3

MODEL ARCHITECTURE (CUSTOM CLASSIFIER HEAD)

Layer Type	Configuration
Input Layer	256 × 256 × 3 RGB image
Base Model	MobileNetV3Small (include_top=False, pretrained on ImageNet)
Dense Layer	256 units, ReLU activation, L2 regularization
Dense Layer	128 units, ReLU activation
Batch Normalization	-
Dropout	30%
Dense Layer	64 units, ReLU activation
Dense Layer	32 units, ReLU activation
Batch Normalization	-
Dropout	10%
Output Layer	Dense(5), softmax activation for 5-class classification

The model was compiled using the Adam optimizer with a learning rate of 0.00001. The loss function used was sparse categorical crossentropy, which is suitable for multi-class classification tasks with integer-labeled data. The training was carried out with a batch size of 16, and the model was trained for a maximum of 70 epochs, though early stopping criteria were applied. Two training callbacks were configured: EarlyStopping, which halts training if validation loss does not improve over several epochs, and ReduceLROnPlateau, which dynamically lowers the learning rate when performance stagnates. These helped prevent overfitting and encouraged efficient convergence.

Table 4 shows the training configuration.

TABLE 4

TRAINING CONFIGURATION

Parameter	Value / Description
Optimizer	Adam
Initial Learning Rate	0.00001
Learning Rate Adjustment	ReduceLROnPlateau (factor=0.5, patience=6)
Minimum LR	1e-7
Loss Function	Sparse Categorical Crossentropy
Batch Size	16
Max Epochs	70 Epochs
Callbacks	EarlyStopping (patience=8), ReduceLROnPlateau (patience=6)
Fine-Tuned Layers	Last 80 layers of MobileNetV3Small
Input Image Size	256 × 256 × 3

This model development approach effectively combined transfer learning, fine-tuning, and regularization strategies to produce a lightweight and accurate classifier for tomato leaf disease detection. By balancing performance with efficiency, the model was optimized for deployment across both mobile and web-based platforms.

E. System Development

The proposed system was designed to function as a web-app that allows users to upload tomato leaf images for automatic disease classification using a trained Convolutional Neural Network (CNN) model. In addition to image analysis, the system features a forum for discussion, a prediction record log for users to track past diagnoses and users can access AI-generated insights powered by Ollama through JavaScript-based API calls. The system's creation was comprehensive, encompassing the frontend, backend, model deployment, and database administration.

TABLE 5

SYSTEM DEVELOPMENT

System Component	Technology Used
Frontend	HTML, TailwindCSS, JavaScript (used to fetch Ollama API)
Backend	PHP (for logic and database interaction)
Database	MySQL (via XAMPP)
Model Deployment	Keras
Hosting Environment	Localhost (via XAMPP)

The user-side interface, crafted with modern web technologies, offers a smooth experience. HTML lays out the content, TailwindCSS ensures a responsive and polished look, and JavaScript adds dynamic features like instant image previews and seamless form submissions. Users can easily upload tomato leaf images for disease classification, navigate forum posts, and view their past prediction records. JavaScript also fetches AI-generated insights via API calls to Ollama, enhancing the information shown to the user after a prediction.

The system's backend, built with PHP, manages all user interactions, server-side operations, and database communication. It handles essential functions like user authentication, processing forum submissions, and storing prediction results from the machine learning model in a MySQL database (managed via XAMPP). This backend ensures smooth data flow across the entire system.

The tomato disease classification model, developed with Keras, is seamlessly integrated. When a user uploads an image, the backend sends it to the CNN model for analysis. The model then identifies any leaf disease and returns the prediction, which is immediately stored and shown to the user.

For streamlined development and testing, all system components run on a local server powered by XAMPP. This package conveniently integrates Apache, PHP, and MySQL, providing a unified platform where the entire full-stack application can function cohesively before its final deployment.

The system gains significant value from its Ollama integration, providing AI-powered explanations or treatment advice following disease classification. Using JavaScript's async/await with the fetch() function, the client queries the Ollama backend to retrieve relevant insights. These insights are then dynamically displayed on the frontend, offering immediate, intelligent feedback to the user.

This architecture was chosen to ensure a clear separation of concerns, optimize performance and modularity, and facilitate future expansion. This design readily supports the addition of new crop types, enhanced community features, or a seamless transition to cloud-based deployment.

F. Evaluation Metrics

To evaluate the performance of the models in classifying grapes and non-grapes, several standard classification metrics were employed. These include accuracy, precision, recall, F1 score, and confusion matrix. In addition, to better understand model decision-making, Grad-CAM (Gradient-weighted Class Activation Mapping) was utilized to visually interpret model focus areas during prediction. These evaluation strategies ensure not only performance quantification but also explainability, which is crucial for trust and deployment in real-world agricultural applications.

1. Accuracy

Accuracy represents the proportion of correct predictions (both true positives and true negatives) out of the total predictions. It provides a general performance overview and is calculated as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

2. Precision

Precision measures the reliability of positive predictions by calculating the ratio of true positives to all samples predicted as positive. It highlights the model's ability to avoid false positives and is defined as:

$$\text{Precision} = \frac{TP}{TP + FP}$$

3. Recall

Recall, or sensitivity, evaluates the model's ability to correctly identify all relevant positive instances. It calculates the proportion of true positives out of all actual positives, offering insight into the model's capacity to minimize false negatives:

$$\text{Recall} = \frac{TP}{TP + FN}$$

4. F1 Score

The F1 Score combines Precision and Recall into a single metric by calculating their harmonic mean, which is beneficial in cases where both false positives and false negatives are critical to performance. This balanced measure is defined as:

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

5. Confusion Matrix

For this study, the confusion matrix is used to evaluate the performance of the classification models across five distinct classes: Bacterial Spot, Healthy, Leaf Mol', Septoria Leaf Spot, and Yellow Leaf Curl Virus.

A multi-class confusion matrix displays a square matrix where each row represents the actual class and each column represents the predicted class. The diagonal values indicate the number of correct predictions for each class, while the off-diagonal values show misclassifications.

TABLE 6

CONFUSION MATRIX

Actual \ Predicted	Bacterial Spot	Healthy	Leaf Mold	Septoria Leaf Spot	Yellow Leaf Curl Virus
Bacterial Spot	TP				
Healthy		TP			
Leaf Mold			TP		
Septoria Leaf Spot				TP	
Yellow Leaf Curl Virus					TP

- Diagonal cells (TP): Correct predictions for each class
- Off-diagonal cells: Misclassifications (e.g., instances of 'Leaf Mold' predicted as 'Bacterial Spot')

This matrix enables a detailed evaluation of model errors, showing which specific diseases are frequently confused with others. For instance, if 'Leaf Mold' is often predicted as 'Septoria Leaf Spot', it may suggest the visual features between those two are similar, prompting further review of training data or model tuning.

To supplement the numeric confusion matrix, visualizations (e.g., using matplotlib and seaborn in Python) are typically generated as heatmaps for intuitive interpretation.

6. Interpretability using Grad-CAM

To visualize and interpret the decision-making process of the trained models, Gradient-weighted Class Activation Mapping (Grad-CAM) was applied to the final convolutional layer of MobileNetV3Small.

Grad-CAM generates heatmaps that highlight image regions most influential to the model's prediction. By overlaying these heatmaps on the original input images, the researchers assessed whether the model was focusing on disease-specific features such as: Leaf discolorations, Spot patterns, Edge anomalies, and Vein structures.

Overall, the evaluation metrics and interpretability techniques employed in this study provided a comprehensive understanding of the model's performance and behavior. Quantitative metrics such as accuracy, precision, recall, F1 score, and the confusion matrix allowed for a detailed assessment of classification effectiveness across all five disease classes. Meanwhile, Grad-CAM visualizations offered critical insights into the spatial focus of each model, enhancing transparency and interpretability. Together, these tools not only validated the reliability of the deep learning models in diagnosing tomato leaf diseases but also highlighted areas for improvement, ultimately contributing to the development of more accurate and trustworthy AI-assisted agricultural diagnostics.

III. RESULTS AND DISCUSSION

This section presents and analyzes the performance outcomes of the developed tomato leaf disease classification model under two training approaches: without fine-tuning and with fine-tuning of the base MobileNetV3Small architecture. The evaluation focuses on key performance metrics, including accuracy, loss, precision, recall, and F1-score, as well as visual tools such as training curves, confusion matrices, and Grad-CAM visualizations to assess learning behavior and model interpretability.

Through these comparative analyses, the effectiveness of applying fine-tuning and early stopping strategies is explored, highlighting their impact on both learning efficiency and classification accuracy. The results serve to validate the model's suitability for deployment in real-world agricultural scenarios, particularly in mobile and web-based applications.

A. Training Performance Comparison

Table 6 shows the training and validation performance metrics of the non-fine-tuned model.

TABLE 7

TRAINING AND VALIDATION PERFORMANCE METRICS OF THE NOT FINE-TUNED MODEL

Training Performance (70 Epochs)	
Highest Training Accuracy	0.83
Highest Validation Accuracy	0.79
Lowest Training Loss	0.50
Lowest Validation Loss	0.59
Final Training Accuracy	0.83
Final Validation Accuracy	0.79
Final Training Loss	0.50
Final Validation Loss	0.59

The training performance of the tomato leaf disease classification model was evaluated under two configurations: without fine-tuning and with fine-tuning. In the first setup, the model was trained for 70 epochs without unfreezing any base layers of the MobileNetV3Small architecture. It achieved a maximum training accuracy of 83% and validation accuracy of 80%, with corresponding training and validation losses of 0.51 and 0.55, respectively. These metrics remained unchanged from their peak to final values, indicating that the model had plateaued early and reached its learning capacity.

TABLE 8

TRAINING AND VALIDATION PERFORMANCE METRICS OF THE FINE-TUNED MODEL

Training Performance (48 Epochs)	
Highest Training Accuracy	0.99
Highest Validation Accuracy	0.90
Lowest Training Loss	0.12
Lowest Validation Loss	0.34
Final Training Accuracy	0.99
Final Validation Accuracy	0.90
Final Training Loss	0.12
Final Validation Loss	0.35

In contrast, the fine-tuned model, which unfroze the top 80 layers of the base network, exhibited significantly improved performance despite being early-stopped at epoch 48. It achieved a highest training accuracy of 99%, a final training accuracy of 99%, and a validation accuracy of 90%. The loss values were also considerably lower, with training loss at 0.12 and validation loss at 0.35. This suggests the fine-tuned model achieved better convergence, faster learning, and stronger generalization within fewer epochs.

B. Training Curves and Learning Dynamics

The training and validation curves further illustrate the learning dynamics of both models. For the not fine-tuned model, the curves show a slow, steady rise in accuracy and a plateauing trend around the mid-epochs, typical of a model reaching limitations in architecture or feature extraction. The persistent gap between training and validation accuracy suggests a modest generalization capability.

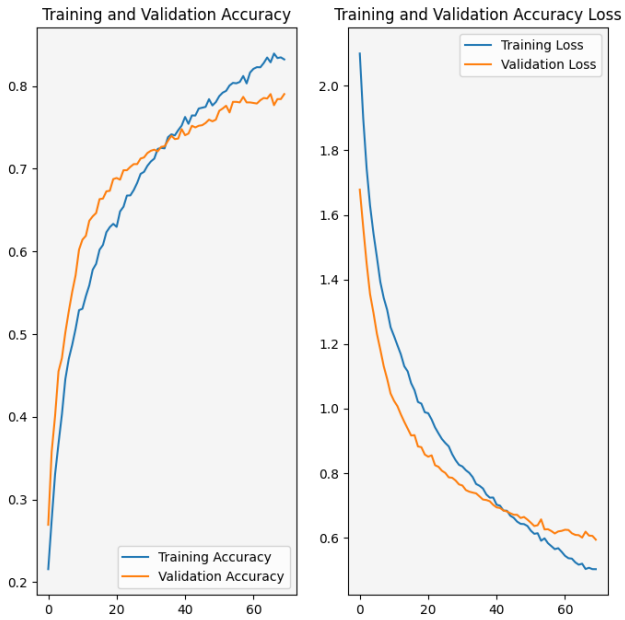

Fig. 2 Not Fine-Tuned Plotting

Conversely, the fine-tuned model's curves demonstrate a sharper incline in performance, stabilizing earlier while maintaining a narrower gap between training and validation metrics. The early stopping at epoch 48 reflects optimal convergence, helping to prevent overfitting and preserve generalization.

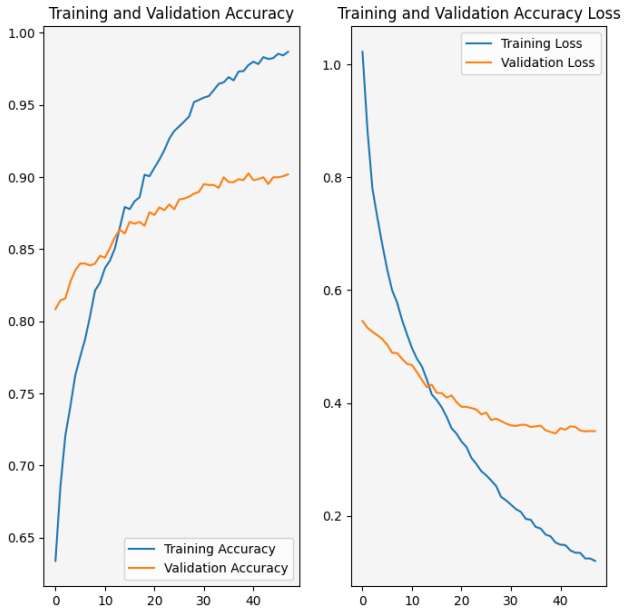

Fig. 3 Fine-Tuned Plotting

C. Evaluation Metrics

To assess classification performance beyond accuracy alone, precision, recall, and F1-score were computed. The non-fine-tuned model achieved 82% for F1-Score, 84% for precision, and 81% for recall and overall accuracy, indicating balanced but moderate performance. Meanwhile, the fine-tuned model achieved 90% for precision, recall, F1-score, and overall accuracy.

TABLE 9

COMPARISON OF EVALUATION METRICS

	F1-Score	Precision	Recall	Accuracy
Not Fine-Tuned	0.82	0.84	0.81	0.81
Fine-Tuned	0.90	0.90	0.90	0.90

The consistent improvement across all metrics confirms that fine-tuning enhanced the model's discriminative power without favoring any specific performance aspect, resulting in a more robust and reliable classification system.

D. Confusion Matrix Analysis

The confusion matrices provide a class-wise view of model accuracy. Figure 4 shows the confusion matrix of the not fine-tuned.

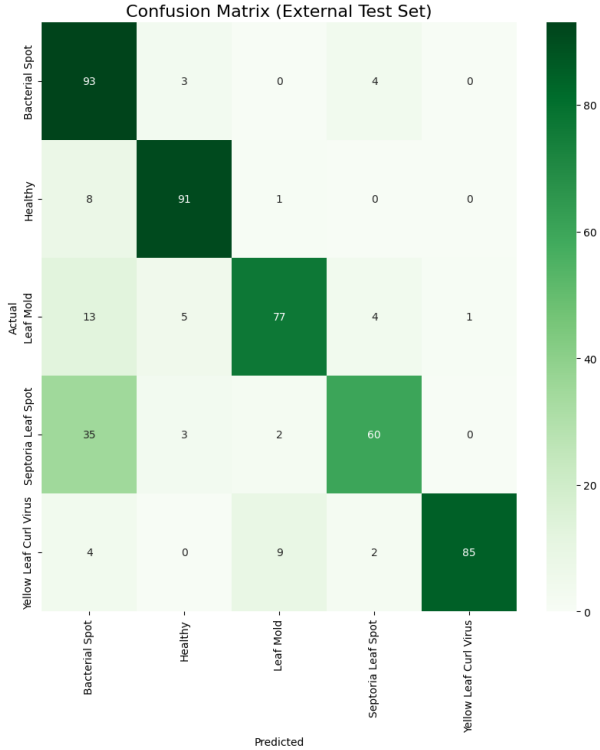

Fig. 4 Not Fine-Tuned Confusion Matrix

The not fine-tuned model exhibits moderate diagonal values, with misclassifications distributed across multiple classes. This indicates that while the model could distinguish some disease types, its confidence and precision were limited.

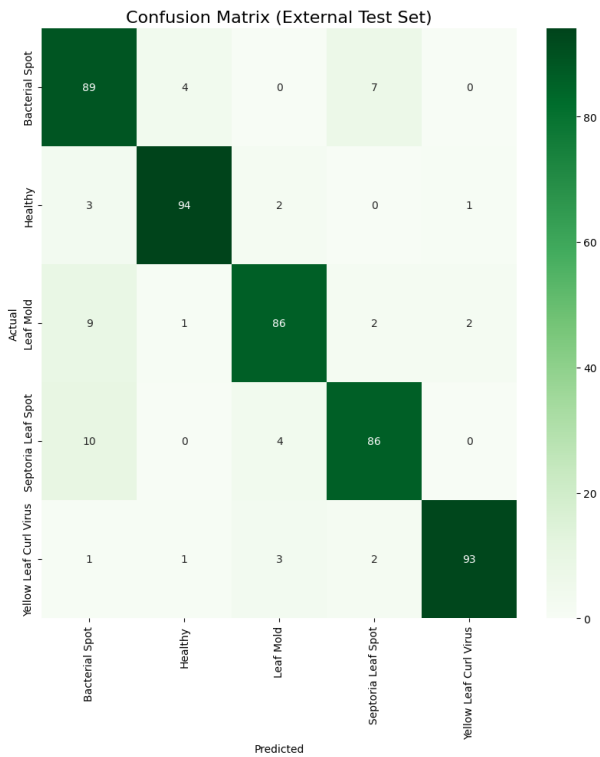

Fig. 5 Fine-Tuned Confusion Matrix

In contrast, the fine-tuned model displays a stronger diagonal dominance, reflecting higher confidence and accuracy in classifying each tomato leaf condition. Misclassifications are notably reduced, which aligns with the improvement in evaluation metrics and supports the claim of improved model generalization.

E. Model Interpretability with Grad-CAM

To ensure the model focused on meaningful features during classification, Grad-CAM visualizations were generated. These heatmaps highlight the specific regions of each input image that influenced the model's prediction. Figure 6 shows the model's gradcam

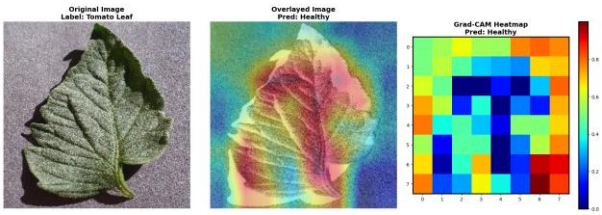

Fig. 6 Model's Gradcam

The Grad-CAM outputs show that the model generally attends to relevant leaf areas with visible symptoms, rather than background noise or irrelevant regions. This adds a layer of interpretability and trustworthiness, confirming that the model learns useful, disease-related patterns rather than spurious correlations.

IV. CONCLUSIONS

This study successfully developed and evaluated a deep learning-based tomato leaf disease classification system using the MobileNetV3Small architecture. Through a developmental-experimental research design, the model was trained on a balanced dataset across five tomato leaf classes and deployed on both web and mobile platforms for practical use in real-world agricultural settings.

The fine-tuned model, which incorporated transfer learning and selective unfreezing of the top 80 layers, significantly outperformed the non-fine-tuned version. It achieved a validation accuracy of 90%, an improvement of 8% over the baseline, and demonstrated stronger generalization with lower loss values and better class-wise performance as indicated by the confusion matrix and performance metrics. Additionally, Grad-CAM visualizations confirmed the model's ability to focus on disease-relevant leaf regions, enhancing interpretability.

The integration of the trained model into a cross-platform application ensures that users — particularly farmers and agricultural workers — can access real-time, AI-driven disease detection tools even in resource-limited environments. The consistent improvement in metrics, reduced training time, and deployment efficiency make the fine-tuned model highly suitable for practical, field-ready deployment in precision agriculture.

REFERENCES

Durmus, H., Gunes, E. O., & Kirci, M. (2017). Disease detection on the leaves of the tomato plants by using deep learning. 2017 6th International Conference on Agro-Geoinformatics, 1-5.

Gugino, B. K., Abawi, G. S., & Ludwig, J. W. (2018). Septoria leaf spot of tomato. Cornell Cooperative Extension. Plant Disease Diagnostic Clinic.

Hukkeri, M. S., Shivakumar, B. L., & Suresh, K. V. (2024). Classification of various plant leaf diseases using pretrained convolutional neural networks on ImageNet. International Journal of Advanced Computer Science and Applications, 15(3), 234-242.

Ivey, M. L. (2023). Septoria leaf spot. NC State Extension Plant Pathology. https://plantpathology.ces.ncsu.edu/plantpathology-factsheets/septoria-leaf-spot/

Mahmud, S., Rahman, M. M., Fattah, S. A., Funayama, H., & Siddique, N. (2024). Light-weight deep learning model for accelerating the classification of mango-leaf disease. Neural Computing and Applications, 36, 4327-4342.

Rangarajan, A. K., Purushothaman, R., & Ramesh, A. (2018). Tomato crop disease classification using pre-trained deep learning algorithm. Procedia Computer Science, 133, 1040-1047.

Sánchez-Campos, S., Martínez-Ayala, A., Márquez-Martín, B., Aragón-Caballero, L., Navas-Castillo, J., & Moriones, E. (2021). Fulfilling Koch's postulates confirms the monopartite nature of tomato leaf deformation virus. Virus Research, 292, 198220.

Zhang, S., Liu, S., Alavez, K. E. Z., Cai, F., Wang, R., Liu, T., ... & Liang, G. (2022). Occurrence, distribution, and management of tomato yellow leaf curl virus in China. Phytopathology Research, 4(1), 1-18.